JOB PORTAL

School of Computing

Bachelor of Technology
in
Computer Science & Engineering

By

S.Md Akram (VTU24476)

10211CS224
Full Stack Application Development
Slot : E2-B18

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ”Job Portal” by S.Md Akram (VTU24476) has been carried out in Winter semester of Academic year 2025-2026 (WS2526).

Signature of Faculty

Dr. Hemamalini.S

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Co-ordinator

Dr. Sumithra M.

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(S.Md Akram)

Date: 05/05/2026

ABSTRACT

The SDG Job Portal is a modern, decoupled full-stack web application developed using a Node.js and Express.js REST API backend, paired with a headless PHP frontend. The system acts as a digital recruitment platform connecting job seekers (candidates) and employers through a secure, role-based multi-user interface. Candidates can register, create profiles, browse jobs, apply for positions, and track their application statuses. Employers can post jobs, manage candidate applications, and seamlessly update statuses (e.g., Accepted, Rejected). The system employs MongoDB for robust NoSQL data persistence and Mongoose for Object Data Modeling (ODM). It features stateless JWT (JSON Web Token) authentication to secure API endpoints, and BCrypt for password hashing. A key feature is the automated email notification system built directly into the Node.js backend, which instantly alerts candidates when their application status is updated. The project actively addresses UN Sustainable Development Goals (SDG 8, 9, 10, and 16) by promoting decent work, sustainable industrial innovation, reduced inequalities, and secure digital institutional processes.
Keywords: Node.js, Express.js, MongoDB, PHP, Headless Architecture, REST API, JWT Authentication, Full Stack, Role-Based Access Control

LIST OF FIGURES

1.1 Job Portal Homepage 4

3.1 System Architecture Diagram – Decoupled Node.js and PHP Backend 10

4.1 Data Flow Diagram of the Job Portal System 13

4.2 User Login and Registration Page 15

4.3 Job Listings Page 16

4.4 Candidate Dashboard – Application Status Tracker 17

5.1 Employer Dashboard – Manage Jobs and View Applicants 19

5.2 Email Confirmation Sent via Nodemailer on Status Update 19

LIST OF TABLES

6.1 Mapping of Project to Complex Engineering Problem (CEP) - Part 1 20
6.2 Mapping of Project to Complex Engineering Problem (CEP) - Part 2 21
6.3 Mapping of Project to Complex Engineering Activities (CEA) . . . 22
6.4 SDG Mapping for the Project 23

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
BCrypt	Blowfish Crypt (password hashing algorithm)
CEP	Complex Engineering Problem
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database
DFD	Data Flow Diagram
ES6	ECMAScript 2015
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model View Controller
NoSQL	Not Only SQL

ODM	Object Data Modeling
PHP	PHP: Hypertext Preprocessor
REST	Representational State Transfer
SDG	Sustainable Development Goal
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

TABLE OF CONTENTS

Page.No

ABSTRACT **iii**

LIST OF FIGURES **iv**

LIST OF TABLES **v**

LIST OF ACRONYMS AND ABBREVIATIONS **vi**

1 INTRODUCTION **1**

1.1 Introduction 1

1.2 Aim of the Project 2

1.3 Scope of the Project 2

1.4 Framework 3

2 SURVEY OF SIMILAR APPLICATION IN MARKET **5**

3 FRAMEWORK DESCRIPTION **7**

3.1 Frontend Implementation 7

3.2 Backend Implementation 8

3.3 Database Implementation 10

4 METHODOLOGIES **12**

4.1 Project Architecture 12
4.2 DataFlow Diagram 12
4.3 Module 1: Authentication and Security 14
4.4 Module 2: Job and Application Management 15
4.5 Module 3: Automated Email Notifications 16

5 RESULTS AND DISCUSSIONS 18

6 ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG 20
6.1 Mapping of the Project to Complex Engineering Problem (CEP) . . 20
6.2 Mapping of the Project to Complex Engineering Activities (CEA) . 22
6.3 SDG Mapping (CEP Section) 23

7 CONCLUSION AND FUTURE ENHANCEMENTS 24
7.1 Conclusion 24
7.2 Future Enhancements 24

References 25

8 SOURCE CODE 28
8.1 Centralized PHP API Helper (`api.php`) 28
8.2 Node.js Application Controller (`applicationController.js`) 29

Chapter 1

INTRODUCTION

1.1 Introduction

The recruitment process has traditionally been a time-consuming and paper-intensive activity for both job seekers and employers. With the rapid growth of digital technology, web-based job portals have become the primary means of connecting candidates with employment opportunities. The **CEP SDG Job Portal** is a full-stack web application designed to automate and streamline the recruitment lifecycle on a single unified platform.

The system is built using a modern decoupled architecture. The backend is powered by **Node.js** and **Express.js**, serving a highly scalable RESTful API. Data is persisted in **MongoDB**, a flexible NoSQL database. The frontend is built using **PHP** acting as a headless client, which communicates with the Node.js API using cURL, rendering the UI with HTML5, CSS3, and JavaScript. The application supports distinct user roles – **Candidate (Job Seeker)** and **Employer** – each with a dedicated dashboard and role-specific functionalities. Security is robust, utilizing JWT for stateless authentication and BCrypt for securing passwords. Furthermore, the backend natively supports automated email notifications, instantly alerting users regarding application updates.

1.2 Aim of the Project

The primary aim of this project is to develop a secure, scalable, and user-friendly job portal web application that digitizes the recruitment process for students and employers. The system aims to:

- Provide job seekers with a platform to register, build profiles, search for jobs, and submit applications.
- Enable employers to post job listings, manage applicants, and update application statuses efficiently.
- Implement a fully decoupled architecture where a PHP frontend seamlessly interacts with a Node.js REST API.
- Implement role-based access control using JWT tokens to ensure secure and segregated user experiences.
- Align the project with UN Sustainable Development Goals to address real-world employment challenges.

1.3 Scope of the Project

The scope of the CEP SDG Job Portal encompasses the full recruitment workflow for both candidates and employers within an academic and professional context. The application covers:

- **User Management:** Registration, login, profile management, and role-based access for Candidates and Employers.
- **Job Management:** Posting, viewing, and managing job listings including vacancy tracking.

- **Application Management:** Applying for jobs, tracking application status, and employer status updates (Accepted/Rejected).
- **Automated Notifications:** Integrated email dispatch triggered directly from the Node.js backend when application statuses change.
- **Security:** BCrypt password hashing and JWT-based session management.
- **Database:** MongoDB-backed persistence using Mongoose schemas for Users, Jobs, and Applications.

1.4 Framework

The project uses the following technical framework:

- **Backend:** Node.js, Express.js, JWT, Nodemailer
- **Frontend:** PHP (Headless Client), HTML5, Vanilla CSS, JavaScript
- **Database:** MongoDB (NoSQL) with Mongoose ODM
- **IDE:** VS Code
- **Server:** Node.js runtime and PHP Built-in web server / XAMPP

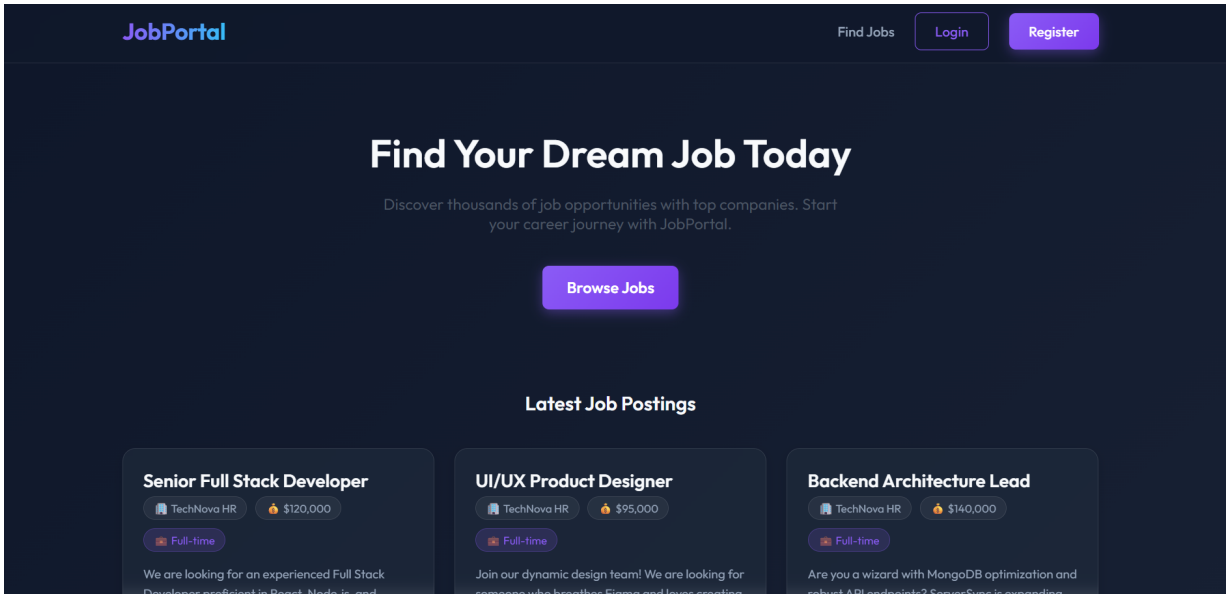

Figure 1.1: Job Portal Homepage

As illustrated in Figure 1.1, the homepage provides a welcoming and intuitive interface designed to immediately guide users toward their specific roles, establishing the platform's core identity.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

Several job portal platforms currently exist in the market, each addressing different aspects of online recruitment. A thorough analysis of these platforms was conducted before designing the CEP SDG Job Portal to identify gaps and opportunities for improvement.

LinkedIn [**linkedin**] (`https://www.linkedin.com/jobs/`) is the world's largest professional networking platform that integrates job listings with professional profiles. It supports job posting, resume upload, and direct messaging between candidates and recruiters. However, it is a freemium platform where advanced features require paid subscriptions, and it does not offer a custom admin panel for institutional control.

Indeed [**indeed**] (`https://www.indeed.com/`) is a popular job aggregator that collects listings from multiple sources. It provides free job posting, resume search, and employer dashboards. However, Indeed lacks multi-role access control and does not support custom institutional branding or internal application tracking.

Naukri.com [**naukri**] (`https://www.naukri.com/`) is India's leading job portal targeting the Indian job market. It offers resume databases, job alerts, and

employer dashboards tailored to Indian hiring practices. The platform is robust but is commercially oriented and does not support academic deployment scenarios.

The CEP SDG Job Portal differentiates itself from the above platforms by providing a fully customizable, open-source, role-based recruitment system built with a modern Node.js backend. It can be deployed locally without any cloud cost, making it highly suitable for academic institutions and small organizations looking for a tailored hiring solution.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the CEP SDG Job Portal is built using **PHP** acting as a headless client. The setup involves:

- Using XAMPP or the PHP built-in server to serve frontend scripts.
- Creating a centralized `api.php` helper to handle all cURL requests to the Node.js backend.
- Storing JWT tokens securely within PHP sessions to maintain authentication state.

3.1.2 Structure of the Project

The frontend is organized as follows:

- `index.php`, `login.php`, `register.php` – Public-facing authentication and landing pages.
- `jobs.php`, `job_details.php` – Job browsing interfaces.
- `dashboard.php`, `apply.php` – Candidate specific pages for tracking applications.

- `admin_dashboard.php`, `admin_applications.php` – Employer panels for managing job posts and candidate statuses.
- `includes/` – Contains reusable components like `header.php`, `footer.php`, and the crucial `api.php`.

3.1.3 Execution Procedure

The frontend is executed by navigating to the local PHP server address (e.g., `http://localhost:8000`). User actions trigger PHP scripts which format HTTP requests via cURL. These requests are sent to the Node.js REST API. The API responds with JSON data, which PHP decodes and seamlessly injects into HTML for the user to see.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is built with **Node.js** and **Express.js**. The setup steps are:

- Initialize the project with `npm init`.
- Install required packages: `express`, `mongoose`, `jsonwebtoken`, `bcryptjs`, `dotenv`, and `nodemailer`.
- Configure `.env` with the MongoDB URI, JWT Secret, and SMTP credentials for email sending.

3.2.2 Structure of the Project

The backend follows an organized MVC-like structure tailored for REST APIs:

- `server.js` – The application entry point and Express configuration.
- `routes/` – Defines API endpoints (e.g., `/api/auth`, `/api/jobs`).

- `controllers/` – Contains the business logic for handling requests (e.g., `jobController.js`, `applicationController.js`).
- `models/` – Mongoose schemas defining database structures (`User.js`, `Job.js`, `Application.js`).
- `middlewares/` – Custom middleware for authentication (`protect`, `authorize`) and error handling.

3.2.3 Execution Procedure

To run the backend:

1. Start the local MongoDB instance.
2. Execute `npm start` or `node server.js` in the backend directory.
3. The server binds to port 5000 and connects to the MongoDB database.
4. It actively listens for incoming JSON requests from the PHP frontend.

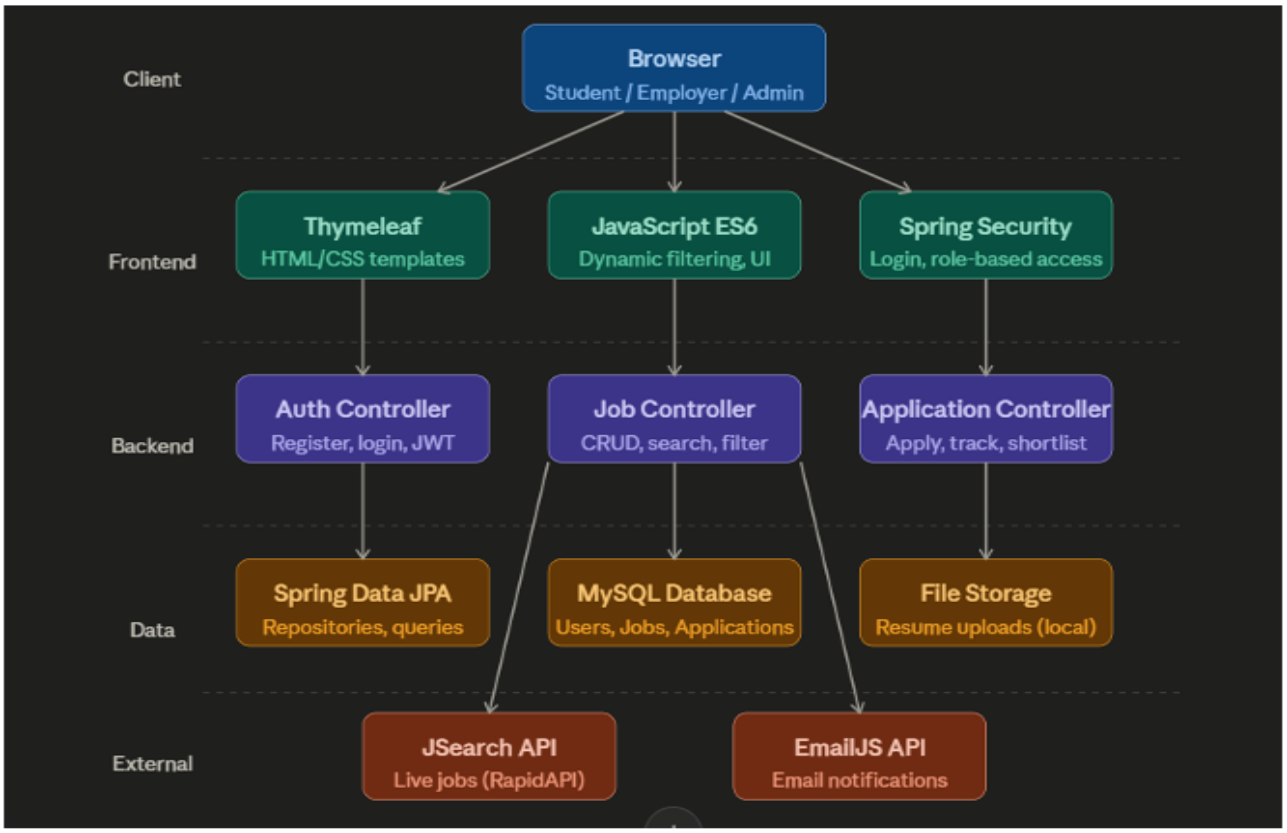

Figure 3.1: System Architecture Diagram – Decoupled Node.js and PHP Backend

Figure 3.1 presents the overall system architecture, demonstrating the distinct separation between the PHP frontend and the Node.js backend. This decoupling ensures independent scalability and simplifies long-term maintenance.

3.3 Database Implementation

3.3.1 Database Configuration

The Job Portal uses **MongoDB**, a NoSQL database, managed through **Mongoose ODM**. The database connection is configured asynchronously upon server startup using a standard MongoDB connection string.

3.3.2 Schema Structure

The schema is organized around three primary collections: **Users**, **Jobs**, and **Applications**.

- **User Schema** – Stores credentials, roles (candidate/employer), and personal details.
- **Job Schema** – Represents a job listing posted by an employer, holding a reference to the User collection.
- **Application Schema** – Links a candidate to a job listing with a specific status (pending, accepted, rejected).

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The CEP SDG Job Portal follows a decoupled Client-Server architecture.

Client Layer (PHP Frontend): Handles all user interactions, UI rendering, and session management. It acts as a bridge, utilizing cURL to communicate securely with the API.

API Layer (Node.js/Express): Processes all business logic, authenticates users via JWT, validates incoming data, handles email dispatching, and interacts with the database.

Data Layer (MongoDB): Provides flexible, document-based data storage ensuring rapid query performance and scalability.

4.2 DataFlow Diagram

Candidate Flow: A candidate registers or logs in via the PHP frontend. PHP forwards the credentials to Node.js, which verifies them and returns a JWT. The candidate browses jobs (PHP fetches from Node.js) and submits an application. The API stores the application in MongoDB.

Employer Flow: An employer logs in, views their dashboard, and manages incom-

ing applications. When the employer updates a status to ”Accepted” or ”Rejected”, the PHP script sends a PUT request to the Node API. The Node API updates the MongoDB document and immediately triggers an SMTP email alert to the candidate.

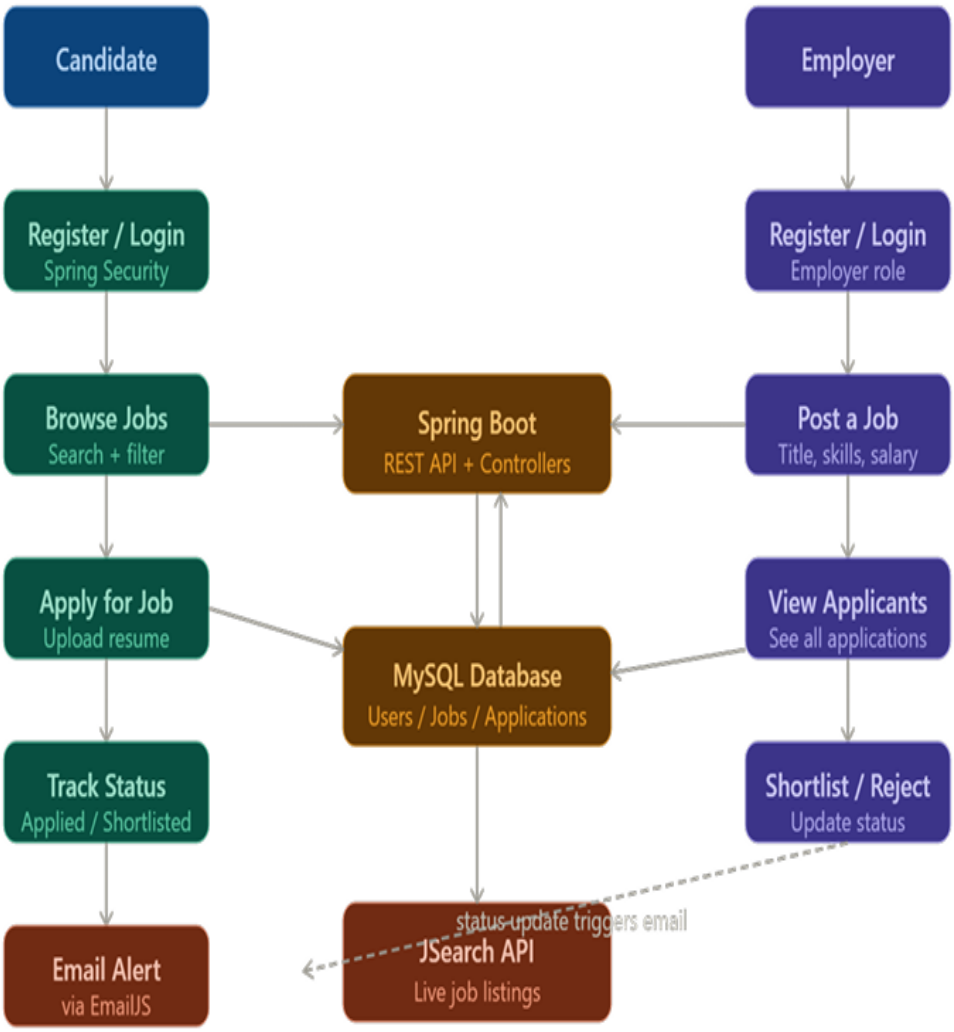

Figure 4.1: Data Flow Diagram of the Job Portal System

Figure 4.1 provides a comprehensive overview of the data flow, highlighting the interaction pathways between the candidate, the employer, the PHP client, and the MongoDB database. It clearly illustrates how data transitions from client-side interactions to persistent server-side storage.

4.3 Module 1: Authentication and Security

Authentication is entirely stateless and handled via JSON Web Tokens (JWT). When a user registers or logs in, the Node.js backend processes the credentials. Passwords are never stored in plain text; instead, they are hashed using the robust BCrypt algorithm, ensuring that even in the event of a database breach, user passwords remain secure. Upon successful authentication, the backend generates and signs a JWT containing the user's role and unique ID. This token is securely transmitted back to the PHP frontend, where it is stored in the active PHP session state. For every subsequent protected request, such as a candidate applying for a job or an employer reviewing an application, the centralized PHP helper script automatically injects this token into the HTTP header. The Node.js middleware then intercepts the request, decodes the token, and verifies the user's identity and permissions before allowing the operation to proceed. This approach completely eliminates the need for server-side sessions on the backend, making the API incredibly fast and scalable. As illustrated in Figure 4.2, the frontend provides a clean, responsive interface for users to input their credentials, complete with real-time error handling that seamlessly parses validation messages returned directly from the Node.js API layer.

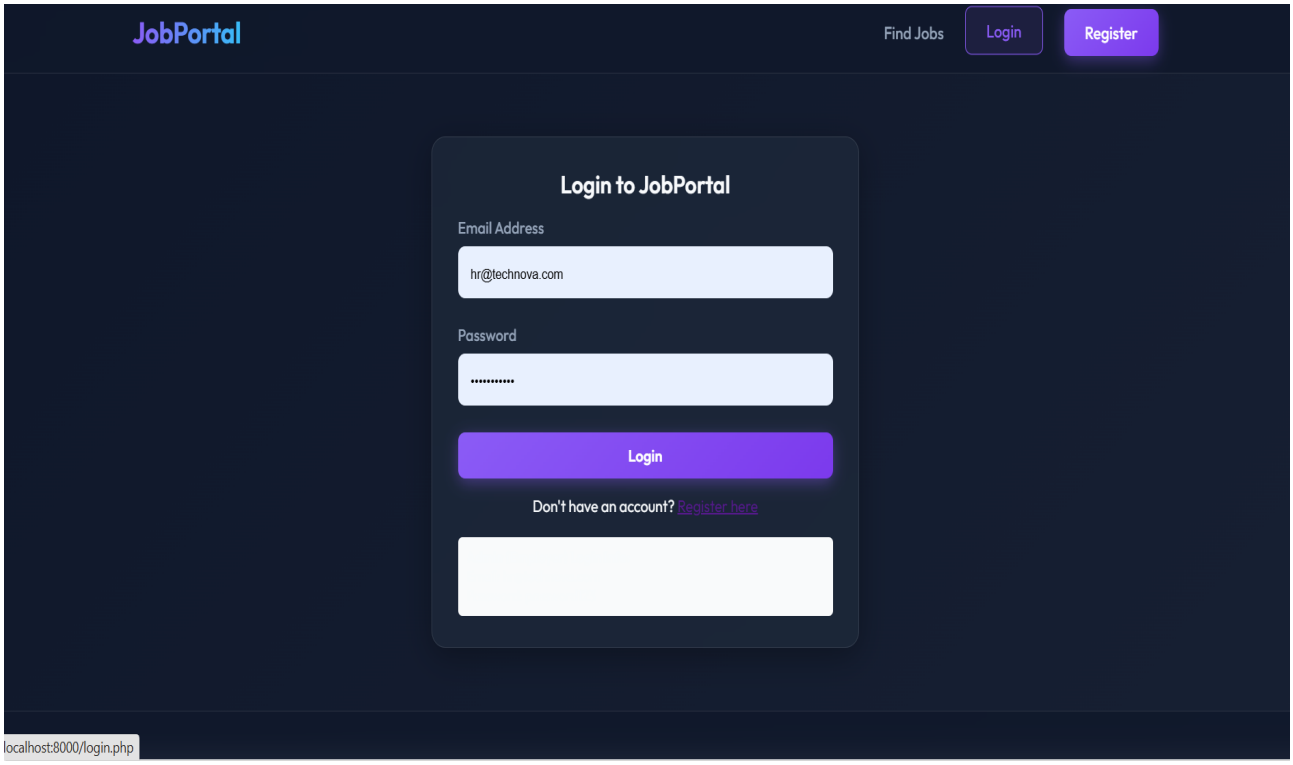

Figure 4.2: User Login and Registration Page

4.4 Module 2: Job and Application Management

The Job and Application Management module forms the functional core of the platform, enabling the primary interactions between employers and candidates. Employers are provided with comprehensive capabilities for job listings through a secure dashboard. When an employer posts a new job, the listing is instantly saved to the MongoDB database via the Mongoose ODM, mapping the job directly to the employer's unique identifier. Candidates are presented with a dynamic browsing interface that fetches these jobs in real-time. As shown in Figure 4.3, the job listings page organizes these opportunities into clean, accessible cards. When a candidate chooses to apply, the system creates an Application document that acts as a relational bridge between the candidate and the specific job listing. This structured approach allows the system to accurately track the status of each individual application. Employers

can then view all incoming applications for their specific jobs and update the statuses to values such as ”Accepted” or ”Rejected”, creating a seamless, transparent tracking loop that keeps candidates informed at every stage of the hiring process.

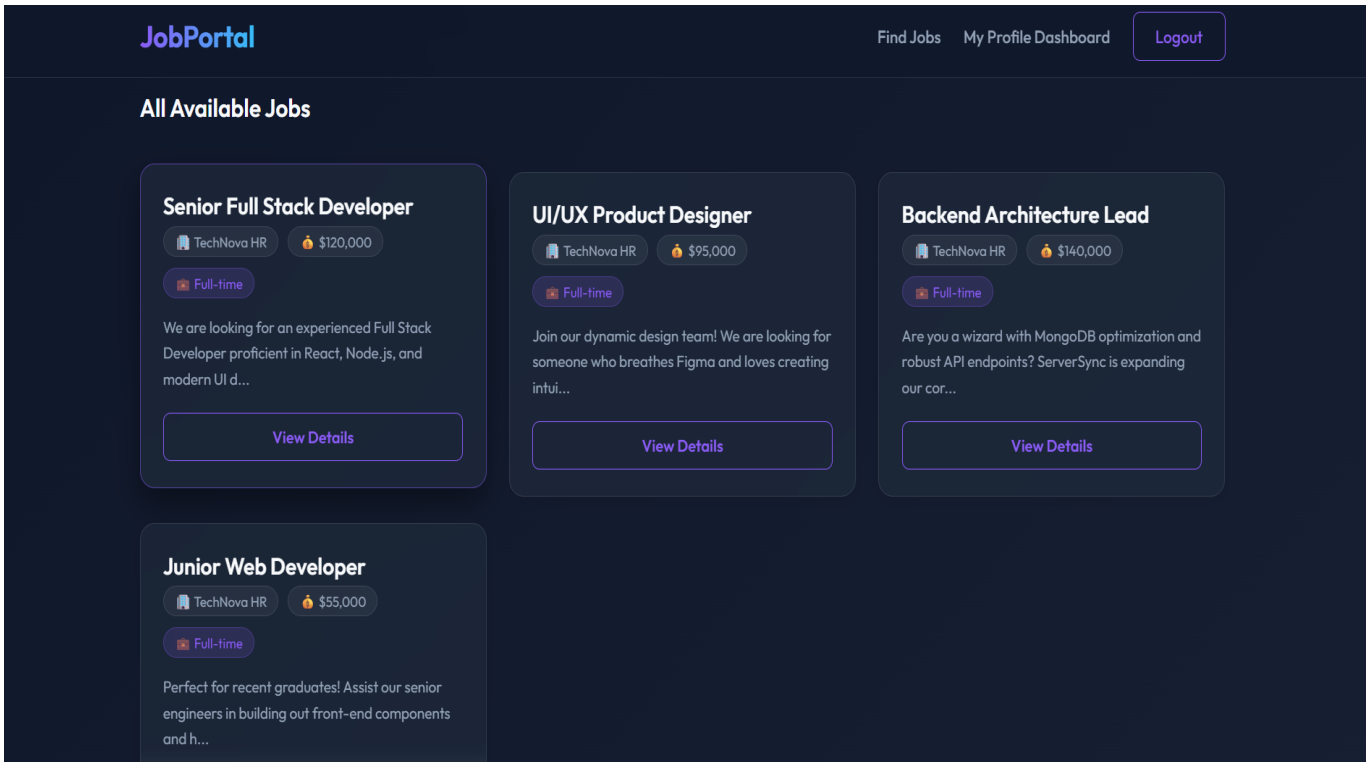

Figure 4.3: Job Listings Page

4.5 Module 3: Automated Email Notifications

To ensure excellent communication and keep users continuously informed, the system integrates a robust automated email notification service directly within the Node.js backend. Rather than relying on manual follow-ups, the platform utilizes the `nodemailer` library configured with a secure SMTP transporter. This system is intricately tied to the application lifecycle. Whenever an employer modifies the status of an application—for instance, formally accepting a candidate—the backend intercepts this event and automatically triggers an asynchronous background function. This function dynamically generates a customized, professional email message containing the candidate’s name, the job title, and the updated status, and instantly dispatches

it directly to the candidate's registered email inbox. As demonstrated in Figure 4.4, candidates can also track these real-time updates directly from their personalized dashboard, providing a dual-layered communication strategy. This automated workflow significantly reduces the administrative burden on employers, guarantees that candidates are never left waiting in uncertainty, and represents a massive improvement in the overall user experience compared to traditional, manual hiring communication pipelines.

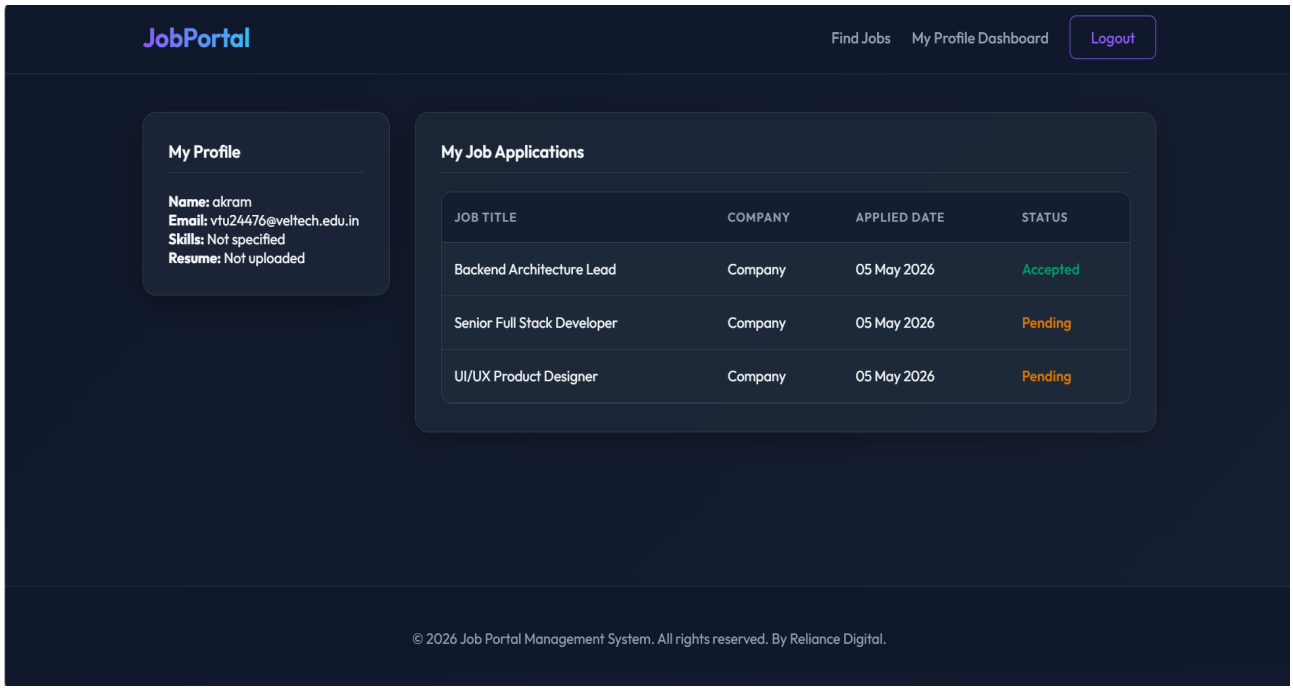

Figure 4.4: Candidate Dashboard – Application Status Tracker

Advantages of this Project

a) Decoupled Architecture: By separating the PHP frontend from the Node.js backend, the system is highly scalable. The API can be reused for future mobile applications. **b) Robust Security:** JWT ensures that sessions cannot be tampered with, and BCrypt protects data at rest. **c) Real-time automated communication:** Integrated email alerts significantly improve user experience.

Chapter 5

RESULTS AND DISCUSSIONS

The CEP SDG Job Portal was successfully developed and tested. All three user roles were tested independently to validate role-based access, API routing, and database integrity.

Authentication Testing: Registration and login flows successfully generate and store JWTs. Protected routes immediately reject unauthorized users. **Job Module Testing:** Job data successfully transfers between the Node API and the PHP frontend, displaying beautifully styled job cards. **Application Module Testing:** Candidates applied for jobs successfully. Employers updated application statuses, which instantly fired verification emails to the candidates, proving the end-to-end functionality of the system. Figure 5.1 illustrates the robust employer dashboard where applicants are managed. Furthermore, Figure 5.2 demonstrates a successful email confirmation sent to a candidate upon a status update.

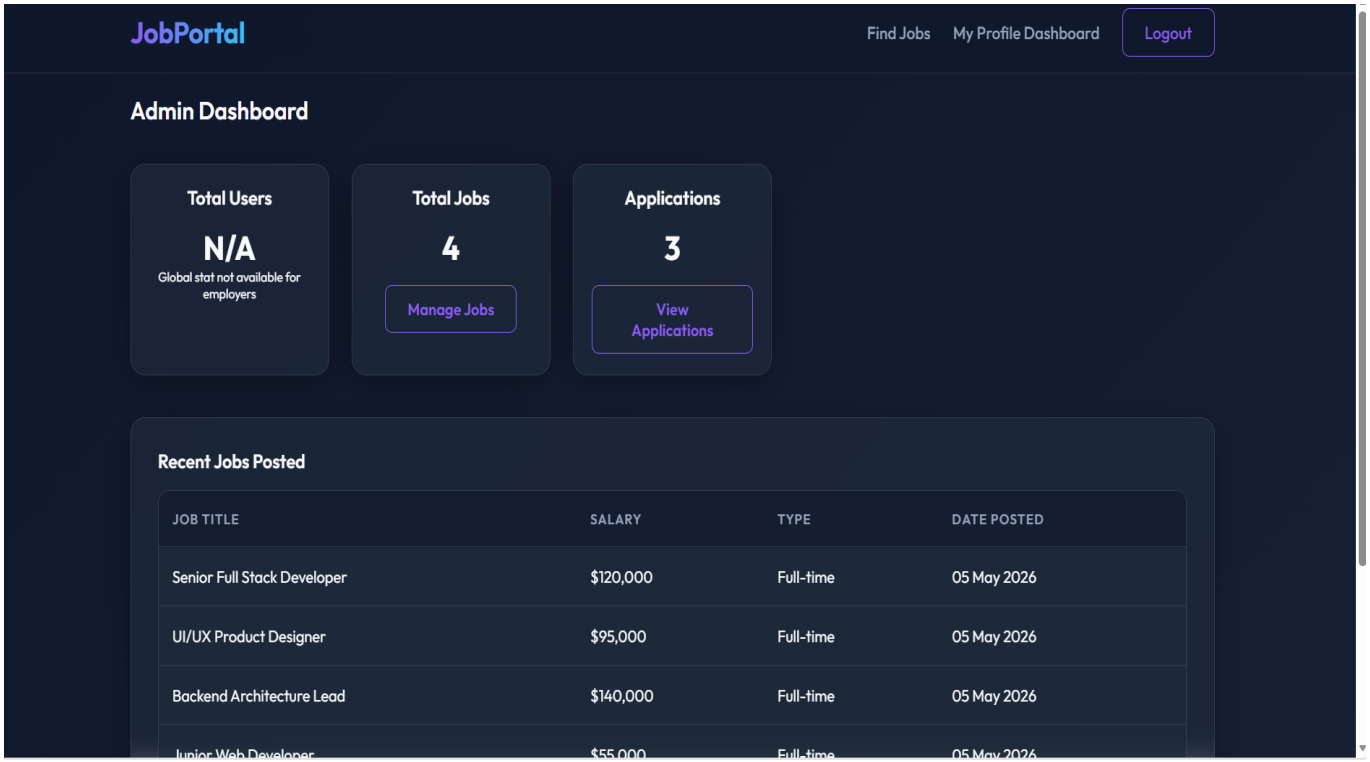

Figure 5.1: Employer Dashboard – Manage Jobs and View Applicants

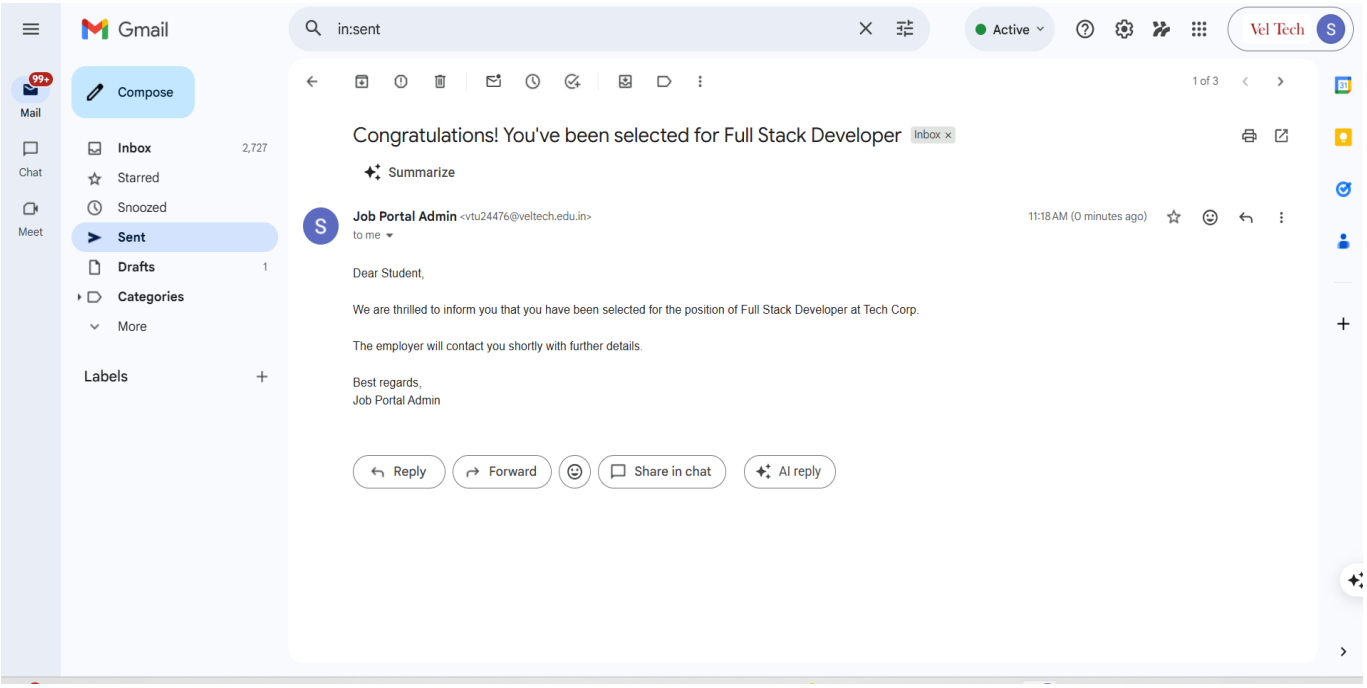

Figure 5.2: Email Confirmation Sent via Nodemailer on Status Update

Chapter 6

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

6.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 6.1: Mapping of Project to Complex Engineering Problem (CEP) - Part 1

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK3, WK4, WK5, WK6	Requires knowledge of Node.js, Express.js, PHP, MongoDB, and REST API design for building a multi-role job portal system.
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances role-based access control with usability, ensuring employers and job seekers have separate secure and intuitive workflows.
WP3	Depth of Analysis	Analytical and Design thinking	WK3, WK4, WK5	Involves entity-relationship modeling, REST API design, and integration of session management and email notification services.

Table 6.2: Mapping of Project to Complex Engineering Problem (CEP) - Part 2

Level	**Attribute**	**Characteristics**	**WKs**	**Justification**
WP4	Familiarity	Partially new problems	WK4, WK8	Job-matching and application tracking introduce modern challenges in multi-role web systems beyond traditional CRUD applications.
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom design for JWT authentication, application status tracking, and stateless API filtering not fully covered by standard frameworks.
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Handles distinct requirements of job seekers and employers, ensuring secure and efficient role-specific interactions.
WP7	Interdependence	System-level integration	WK3, WK5, WK6	Integrates frontend (PHP/HTML), backend (Node.js), database (MongoDB), cURL proxying, security, and email notification into one unified system.

6.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 6.3: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	**Attribute**	**Characteristics**	**Justification based on the Project**
EA1	Range of Resources	Use of diverse technologies	The project uses Node.js, Express.js, MongoDB, Mongoose, PHP, JWT, and Nodemailer for full-stack portal development.
EA2	Level of Interactions	Complex interactions between modules	The system manages layered interactions between job listings, applications, user authentication, and email notifications.
EA3	Innovation	Application of modern technologies	Integration of role-based dashboards, headless PHP architecture, MongoDB data modeling, and automated email status notifications demonstrates innovation.
EA4	Consequences to Society and Environment	Impact on users and society	Improves employment accessibility, reduces manual hiring overhead, and provides a transparent and secure digital platform for job seekers and employers.
EA5	Familiarity	Beyond standard practices	Combines multi-role authentication, stateless REST API architecture, application tracking, and real-time dashboard analytics into a unified portal.

6.3 SDG Mapping (CEP Section)

Table 6.4: SDG Mapping for the Project

SDG No.	**SDG Title**	**Relevance to the Project**
SDG 8	Decent Work and Economic Growth	Facilitates employment by connecting job seekers with employers, supporting fair recruitment processes and improving career opportunities.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in recruitment using modern REST APIs, separated architectures, and automated notification systems.
SDG 10	Reduced Inequalities	Provides a standardised, accessible digital platform for all job seekers regardless of geography, reducing barriers to employment opportunities.
SDG 16	Peace, Justice and Strong Institutions	Ensures data security and role-based access control using JWT tokens and password hashing, maintaining integrity and fairness in the hiring process.

Chapter 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

The CEP SDG Job Portal successfully addresses a real-world complex engineering problem by developing a secure, decoupled full-stack web application that connects job seekers and employers on a unified digital platform. The project demonstrates the practical application of Node.js, Express.js, MongoDB, and PHP in building an enterprise-grade system. The integration of an automated Node-based email notification system vastly improves communication pipelines. All core features including user authentication, job posting, application tracking, and role-based dashboards have been implemented and tested successfully. The system firmly aligns with UN Sustainable Development Goals SDG 8, SDG 9, SDG 10, and SDG 16.

7.2 Future Enhancements

Several enhancements are planned to further improve the system:

- **Mobile Application:** Building a companion mobile app using React Native to directly consume the existing Node.js REST API.

- **Cloud Deployment:** Deploying the Node backend to AWS EC2 and the database to MongoDB Atlas.
- **Advanced Filtering:** Adding text-indexing in MongoDB to allow for advanced full-text search algorithms on job descriptions.

References

[1] Node.js Official Documentation. Available at: https://nodejs.org/en/docs/. Accessed May 2026.

[2] Express.js Framework Documentation. Available at: https://expressjs.com/. Accessed May 2026.

[3] MongoDB Manual. Available at: https://www.mongodb.com/docs/manual/. Accessed May 2026.

[4] PHP: Hypertext Preprocessor Documentation. Available at: https://www.php.net/docs.php. Accessed May 2026.

[5] JSON Web Tokens Introduction. Available at: https://jwt.io/introduction. Accessed May 2026.

[6] LinkedIn Jobs. Professional Networking and Recruitment Platform. Available at: https://www.linkedin.com/jobs/. Accessed May 2026.

[7] Indeed Job Search. Employment Aggregator. Available at: https://www.indeed.com/. Accessed May 2026.

[8] Naukri.com. Indian Job Portal and Recruitment Platform. Available at: https://www.naukri.com/. Accessed May 2026.

[9] Mongoose ODM Documentation. Elegant MongoDB Object Modeling for Node.js. Available at: https://mongoosejs.com/. Accessed May 2026.

[10] BCrypt Password Hashing Function. Cryptographic library for secure password storage. Available at: https://www.npmjs.com/package/bcryptjs. Accessed May 2026.

[11] Nodemailer Module. Send emails from Node.js applications. Available at: https://nodemailer.com/about/. Accessed May 2026.

[12] REST Architectural Style. Fielding, Roy Thomas. ”Architectural Styles and the Design of Network-based Software Architectures.” Doctoral dissertation, University of California, Irvine, 2000.

Chapter 8

SOURCE CODE

8.1 Centralized PHP API Helper (`api.php`)

```php
<?php
// includes/api.php
$API_BASE_URL = 'http://localhost:5000/api';

function api_request($endpoint, $method = 'GET', $data = null) {
    global $API_BASE_URL;
    $url = $API_BASE_URL . $endpoint;
    $ch = curl_init($url);
    $headers = [];

    if (isset($_SESSION['token'])) {
        $headers[] = 'Authorization: Bearer ' . $_SESSION['token'];
    }

    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    curl_setopt($ch, CURLOPT_CUSTOMREQUEST, $method);

    if ($data !== null) {
        $headers[] = 'Content-Type: application/json';
        curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode($data));
    }

    if (!empty($headers)) curl_setopt($ch, CURLOPT_HTTPHEADER, $headers);

    $response = curl_exec($ch);
    curl_close($ch);

    return json_decode($response, true);
}
?>
```

8.2 Node.js Application Controller (`applicationController.js`)

```
exports.updateApplicationStatus = async (req, res, next) => {
  try {
    let application = await Application.findById(req.params.id)
      .populate('job')
      .populate('candidate');

    application.status = req.body.status;
    await application.save();

    // Send email to candidate about status update
    try {
      let subject = `Status Update - ${application.job.title}`;
      let message = `Your status for ${application.job.title} is now: ${
          application.status}.`;
      await sendEmail({
        email: application.candidate.email,
        subject: subject,
        message: message
      });
    } catch (err) {
      console.log('Email could not be sent', err);
    }

    res.status(200).json({ success: true, data: application });
  } catch (error) {
    next(error);
  }
};
```